## 8.5 Preventing SQL injection

If you've spent any serious time developing Web applications, you know of the risk of SQL injection. SQL injection is caused when data is inserted into the database that hasn't been properly sanitized. For example, a classic demonstration of SQL injection would be an exploit that gives a malicious person administrator access to your site.

It is hard to emphasize the importance of preventing SQL injection with your plugin. WordPress is only as secure as its weakest link. It provides many options to access data that do not require any SQL at all. If you can, it is imperative that you use these functions and API methods. You should only use SQL when it's absolutely essential. Using it without proper care can destroy someone's blog. I cannot emphasize this enough.

Say you have an HTML form that requires a username and password. Each of these form fields are named "user" and "pass," respectively. On the backend, the authentication mechanism checks the username and password against the database like this:

```
SELECT * FROM example_usertable WHERE username = '$input_
user' AND password = '$input_pass';
```

If form data is unsanitized but the user actually inputs a proper username and password, then the SQL query works and authentication is granted. However, if someone enters a more malicious string, they can wreak havoc on your system. For example, if the user entered a username of user' OR 1=1;-- and any password, then the SQL will also be valid and will give the user access to the protected system. The query that would actually be run is shown here:

SELECT * FROM example_usertable WHERE username = 'user' OR 1=1;--random password

To understand the practical danger in this query, you have to understand that two dashes (--) is equivalent to a comment. In essence, everything after the -- is ignored by MySQL so it doesn't matter if an invalid password is provided. It could be anything. Also, because you haven't "escaped" the single quote in the username, the single quote becomes the "end" of the value being checked against the username field. Though this is probably invalid as well, it doesn't matter because the attack adds an OR 1=1 (which is always true) into the query. The query reads, "Retrieve the record from the usertable where the username is 'user' or where 1=1." Because 1 always equals 1, the SQL always evaluates as true and an attacker can access the protected resources, application or, possibly, a WordPress plugin.